API Spec: PI Enrichment API

TL;DR

A scheduled internal enrichment service running every 14 days auto-populates and refreshes a PI enrichment cache for all principal investigators. It queries four external data sources—iCite (citation metrics), NIH RePORTER (grant funding), Clarivate JCR (journal prestige), and Google Scholar Metrics (h5-index, h5-median)—aggregating results into a timestamped, persistent cache keyed by PI identifier (last name + department + Wake Forest affiliation). Scoring always reads from this cache, never live API calls. WHO RePORT integration is deferred to v2. No authentication is needed for iCite or NIH RePORTER; Clarivate JCR requires a CLARIVATE_API_KEY stored as an environment variable; Google Scholar Metrics uses a third-party scraping library with no official API.

Goals

Business Goals

- Maintain a refreshed, structured dataset of PI-level research metrics to power the ROI scoring engine.
- Decouple enrichment from scoring, ensuring all ROI calculations use current data without needing live API queries at upload.
- Accumulate historical enrichment data in a structure suitable for future ML model training and analytics.
- Underpin the four ROI scoring categories: Publication Impact, Journal Prestige, Funding Activity, and Research Output.

User Goals

- Allow data managers to verify enrichment data freshness, with visible last-refreshed timestamps.
- Visually flag failed or incomplete enrichments (amber), ensuring data managers are aware of data gaps when making override decisions.
- Provide fast, up-to-date ROI scoring on new or historic datasets, with no wait for live data pulls.

Non-Goals

- The enrichment service does NOT run at upload time or in response to user actions; it's decoupled and scheduled.
- Does NOT fetch publication-level data (handled by a separate PubMed Custom API).
- WHO RePORT international grants data is **deferred** to v2—fields included in schema but not populated at launch.

- Does NOT actually compute ROI scores; the cache serves as infrastructure for the scoring engine, not the scoring engine itself.

User Stories

- **Backend Developer/System**
  - As the enrichment service, I want to query iCite for every PMID in the request_publications table on each cycle, ensuring RCR metrics are always current regardless of publication date.
  - As the enrichment service, I want to query NIH RePORTER by PI name and Wake Forest affiliation, retrieving active grant counts, total award amounts, and follow-on grant signals.
  - As the enrichment service, I want to query Clarivate JCR for all unique ISSNs from PI publication records, returning Impact Factor and quartile.
  - As the enrichment service, I want to try to pull h5-index and h5-median for PI journals via Google Scholar Metrics, failing gracefully if scraping is blocked or unavailable.
- **Team Lead/Admin**
  - As a Team Lead, I want a dashboard showing the last enrichment timestamp, per-source success/failure counts, and a list of flagged PIs, so I can monitor data quality without needing to contact the developer.

Functional Requirements

- **Enrichment Scheduling** (Priority: High)
  - Automatic background job executes every 14 days (configurable by Team Lead; default 14 days, monthly option available).
  - Team Lead can trigger an on-demand manual run from the admin panel.
  - Scheduled enrichment runs during off-peak hours.
- **iCite Integration** (Priority: High)
  - Endpoint: `GET https://icite.od.nih.gov/api/pubs?pmids={pmid_list}` (up to 100 PMIDs per batch).
  - No authentication required.
  - Queries every PMID in request_publications table each cycle—RCR changes over time.
  - Fields: relative_citation_ratio (RCR), citation_count, nih_percentile, authors (author_order), year, title.
  - Aggregation: mean RCR per PI, total citations, mean authors/paper, % papers where PI is first/last author.
  - Rate limit: No official, implement 0.5s delay between batch calls.
  - Retry: Exponential backoff on 429/500/503, max 3 retries.
- **NIH RePORTER Integration** (Priority: High)

    - Endpoint: `POST https://api.reporter.nih.gov/v2/projects/search`
    - No authentication required.
    - Payload includes: pi_names (last+first), org_names ('Wake Forest'), required fields.
    - Filter: active grants only (`ProjectEndDate >= current date`).
    - Fields: active grant count, total award amount, follow-on status (pattern match ActivityCode and ProjectNum).
    - Pagination supported, limit/offset as needed.
    - Rate limit: 1s delay per PI.
    - Retry: Exponential backoff on 429/500/503, max 3 retries.
- **Clarivate JCR Integration** (Priority: High)
    - Endpoint: `GET https://api.clarivate.com/apis/wos-journal/v1/journals/{issn}` (verify exact URL with institutional credentials).
    - Authentication: `CLARIVATE_API_KEY` (env variable, `X-ApiKey` header).
    - Rate limit: 5 requests/second.
    - Fields: Impact Factor (current year), JCR quartile (Q1/2/3/4).
    - Aggregation: Weighted average Impact Factor and quartile (weighted by publication counts per journal per PI).
    - Numeric mapping: quartile to numeric score (Q1=100, Q2=66, Q3=33, Q4=0).
    - Retry: Exponential backoff on 429/500/503, max 3 retries.
    - ISSN not found: mark as missing, log.
- **Google Scholar Metrics Integration** (Priority: High)
    - No official API: retrieve h5-index and h5-median by journal name using third-party library (e.g. scholarly, SerpAPI).
    - High failure risk: Implement 2s minimum delay, random jitter 2–5s.
    - Non-blocking: If data unavailable after max 2 attempts/journal/cycle, mark as missing, journal prestige subscore uses Clarivate only, flag with visible indicator.
    - Log failures: journal name and error type per attempt.
- **PI Enrichment Cache** (Priority: High)
    - Database table: `pi_enrichment_cache`
        - Fields: pi_id, last_name, first_name, department, affiliation, mean_rcr, citation_count, nih_percentile, avg_author_count, first_last_author_pct, active_grant_count, total_award_amount, follow_on_grant, follow_on_grant_verified, impact_factor_avg, jcr_quartile_avg, jcr_quartile_score, h5_index_avg, h5_median_avg, publication_volume_5yr, collab_network_size, last_enriched_timestamp, enrichment_status.
    - All fields NULL-able; primary key `pi_id`.
    - enrichment_status: COMPLETE, PARTIAL, FAILED.

    - last_enriched_timestamp updated on every cycle.
    - Include NULLs instead of zeroes for "missing" vs "true zero".
- **Research Output Computation** (Priority: High)
    - 5-year publication volume: COUNT(DISTINCT pmid) from request_publications WHERE year_published >= (current_year – 5), grouped by pi_id.
    - Collaborative network: COUNT(DISTINCT co_author_last_name) across all PI publications (from iCite authors).
    - Store both values in enrichment cache, updated every cycle.
- **Failure Handling & Flagging** (Priority: High)
    - Per-source status logged: SUCCESS, FAILED, PARTIAL per-PI per-source.
    - enrichment_status logic: COMPLETE (all succeed), PARTIAL (1–3 succeed), FAILED (all fail).
    - UI surface: amber flag for PARTIAL, red for FAILED; tooltip lists source(s) that failed.
    - Admin UI shows cycle results and flagged PIs.
- **WHO RePORT** (Priority: Enhancement, V2)
    - WHO RePORT is not targeted for v1 due to lack of robust API; include placeholders in cache schema for future v2 supplementation.

User Experience

Entry Point & First-Time User Experience

- Enrichment runs in the background; users do not initiate it.
- On first access, data manager/team lead sees data for current PIs, most with recent `last_enriched_timestamp`; newly added PIs may have "pending enrichment" note until next cycle.

Core Experience

- **Step 1:** Data manager views PI Profile or request queue.
    - Each PI record shows a 'Data last refreshed' timestamp (from cache).
    - UI highlights refresh status clearly (COMPLETE, PARTIAL, FAILED).
- **Step 2:** If enrichment failed for a PI, PARTIAL (amber) or FAILED (red) flag appears.
    - Tooltip lists missing data source(s).
- **Step 3:** Team Lead accesses admin dashboard.
    - See history of enrichment cycles: last run, next scheduled run, success rates by source, and list of flagged PIs.
    - Can manually trigger a refill if needed (e.g., after resolving an API outage).
- **Step 4:** At scoring/upload, system reads all enrichment data from cache—no live API calls.

Advanced Features & Edge Cases

- Manual override: Team Lead can trigger an off-schedule enrichment.
- Google Scholar Metrics failures: UI indicates when only Clarivate data is used for Journal Prestige.
- ISSN missing in JCR: Journal flagged in PI Profile with 'Impact Factor unavailable'.
- Enrichment cache fields always NULL when data is missing, not 0, to preserve data integrity.

UI/UX Highlights

- Status flags (amber/red) are high contrast, tooltipped, and clearly explained.
- `last_enriched_timestamp` is always visible in all views that use cache data.
- Responsive: UI does not block or lag during enrichment.
- All failure and partial-enrichment warnings actionable and never silent.

Narrative

At the Wake Forest research analytics center, maintaining accurate, up-to-date PI impact data used to mean laboriously cross-referencing live sources whenever a new project request arrived. This led to delays, timeouts, and constant worries about metrics being out of date. Now, with the PI Enrichment API, everything changes. Every 14 days—without anyone needing to remember or press a button—the enrichment service wakes up, contacting iCite for the latest citation and RCR data, RePORTER for recent grants and follow-on activity, Clarivate JCR for updated Impact Factors, and Google Scholar for the freshest h5-index values it can get. By morning, every PI's enrichment cache is refreshed, and the latest timestamp is visible to the team. When project requests pour in, scoring is instantaneous, powered by the latest intelligence—no waiting, no stale data, and always a clear flag when a metric can't be found. Data managers work confidently, leadership trusts the results, and the institution quietly accumulates a structured, ML-ready database for future innovation.

Success Metrics

- **Enrichment cycle completion rate:** 100% of PIs processed each cycle, status logged for every PI-data source pair.
- **Per-source success rates:** iCite >98%, NIH RePORTER >95%, Clarivate JCR >90%, Google Scholar Metrics >70%.
- **Cache freshness:** No PI enrichment data older than 16 days (14-day cycle + 2-day grace period).
- **Zero silent failures:** Every PI/source logs a status regardless of outcome.
- **Team Lead dashboard accuracy:** 100% agreement between flagged PIs and underlying cache status.

User-Centric Metrics

- Dashboard survey: Data manager confidence in enrichment data ≥4/5 on internal feedback.
- Average time from new request upload to ROI score generation <30 seconds.

Business Metrics

- Reduction in data manager time spent chasing missing metric data (measured by internal time-tracking or survey).
- Improved grant win rate or publication rate due to prioritization by high-fidelity scoring.

Technical Metrics

- Scheduled enrichment task >99% successful run rate.
- <1% error rates for critical cache fields on a per-cycle basis.
- No cache record older than 16 days under normal operation.

Tracking Plan

- Log per-PI, per-source: enrichment attempt datetime, response time, status, fields populated, error codes.
- Log start/end of every enrichment cycle, cycle runtime, and counts of COMPLETE/PARTIAL/FAILED statuses.
- Log all manual enrichment triggers and override actions.

Technical Considerations

Technical Needs

- Background scheduler service for periodic execution (cron or equivalent).
- API client modules for iCite, NIH RePORTER, Clarivate JCR (with API key from environment), and Google Scholar Metrics (third-party library).
- Database/migration for persistent `pi_enrichment_cache` with NULLable fields and enrichment status primary key.
- Admin panel or CLI for manual enrichment triggering and cycle result reporting.

Integration Points

- Authenticated (API key) integration to Clarivate JCR.
- Downstream dependency: all ROI scoring logic reads *only* from `pi_enrichment_cache`.
- Google Scholar Metrics scraping library; must be updatable/swappable.
- PubMed Custom API feeds new PMIDs into request_publications for iCite coverage.

Data Storage & Privacy

- All cache data stored internally; no personal health info.

- API keys (Clarivate) must be stored as environment variables, never in code or config files.
- All fields NULLable; careful separation between 0 (true zero) and NULL (missing data).
- WHO grant placeholder fields present but left empty (NULL) at launch.

Scalability & Performance

- Typical expected PI count: 100's–1,000's (scaling to 10,000+ should be possible).
- Batch sizes and request throttling by source (per requirements) prevent API quota exhaustion.
- Off-peak execution avoids interference with live user actions.
- Google Scholar's fragility is gracefully handled—enrichment never blocks on this source.

Potential Challenges

- NIH RePORTER follow-on grant detection requires precise pattern matching—validate on known records.
- Name matching in RePORTER can produce false positives for common names—implement confidence threshold, flag ambiguous matches.
- Clarivate JCR quotas: Confirm request limits with IT/library before launch.
- Google Scholar Metrics scraping subject to IP blocking—cycle continues, logs all failures for review.
- NULL discipline in schema: All code must differentiate true zero from missing, or scoring may be misleading.

Milestones & Sequencing

Project Estimate

- **Medium:** 1 week (5 working days)

Team Size & Composition

- **Small Team:** 1 backend developer (plus part-time product/design if needed).

Suggested Phases

Phase 1: Scheduler & iCite Integration (Day 1–2)

- Key Deliverables: Background enrichment scheduler, manual trigger (developer); iCite API client and batch-processing; create `pi_enrichment_cache` table.
- Dependencies: Existing request_publications table; API client for iCite.

Phase 2: NIH RePORTER & Clarivate JCR Integration (Day 2–3)

- Key Deliverables: NIH RePORTER client with pattern matching, aggregation logic; Clarivate JCR client and aggregation; per-journal ISSN mapping and weighting.
- Dependencies: Verified Clarivate endpoint and API key from Wake Forest.

Phase 3: Google Scholar Metrics & Output Calculation (Day 3–4)

- Key Deliverables: Google Scholar Metrics scraping module, error handling; output calculations for publication volume and collaborator network.
- Dependencies: Stable Google Scholar scraping library.

Phase 4: Failure Logging & Admin UI (Day 4–5)

- Key Deliverables: Comprehensive enrichment-status logging, enrichment flags in UI, Team Lead dashboard with cycle reporting, manual trigger.
- Dependencies: UI integration, caching logic.

QA & Launch:

- Run one full enrichment cycle on production data, validate with known PI/test cases, review logs and UI flagging for completeness.

Developer Callouts:

1. Confirm Clarivate JCR endpoint/auth flow with portal credentials before Phase 2.
2. Store CLARIVATE_API_KEY only as an env variable.
3. iCite integration must query all PMIDs every cycle, not just new.
4. Enforce NULL/zero discipline in all cache writes and reads.
5. Estimate Clarivate JCR API call volume per cycle; check quota before deployment.
6. Include WHO RePORT placeholder fields in schema to avoid future migration.
